

```
{ return (*check(curr, "dereference past end"))[curr]; }
```

你认为哪个版本更好？为什么？

练习 12.22: 为了能让 `StrBlobPtr` 使用 `const StrBlob`，你觉得应该如何修改？定义一个名为 `ConstStrBlobPtr` 的类，使其能够指向 `const StrBlob`。



12.2 动态数组

`new` 和 `delete` 运算符一次分配/释放一个对象，但某些应用需要一次为很多对象分配内存的功能。例如，`vector` 和 `string` 都是在连续内存中保存它们的元素，因此，当容器需要重新分配内存时（参见 9.4 节，第 317 页），必须一次性为很多元素分配内存。

为了支持这种需求，C++ 语言 and 标准库提供了两种一次分配一个对象数组的方法。C++ 语言定义了另一种 `new` 表达式语法，可以分配并初始化一个对象数组。标准库中包含一个名为 `allocator` 的类，允许我们将分配和初始化分离。使用 `allocator` 通常会提供更好的性能和更灵活的内存管理能力，原因我们将在 12.2.2 节（第 427 页）中解释。

很多（可能是大多数）应用都没有直接访问动态数组的需求。当一个应用需要可变数量的对象时，我们在 `StrBlob` 中所采用的方法几乎总是更简单、更快速并且更安全的——即，使用 `vector`（或其他标准库容器）。如我们将来在 13.6 节（第 470 页）中看到的，使用标准库容器的优势在新标准下更为显著。在支持新标准的标准库中，容器操作比之前的版本要快速得多。



大多数应用应该使用标准库容器而不是动态分配的数组。使用容器更为简单、更不容易出现内存管理错误并且可能有更好的性能。

如前所述，使用容器的类可以使用默认版本的拷贝、赋值和析构操作（参见 7.1.5 节，第 239 页）。分配动态数组的类则必须定义自己版本的操作，在拷贝、复制以及销毁对象时管理所关联的内存。



直到学习完第 13 章，不要在类内的代码中分配动态内存。

12.2.1 new 和数组



为了让 `new` 分配一个对象数组，我们要在类型名之后跟一对方括号，在其中指明要分配的对象数目。在下例中，`new` 分配要求数量的对象并（假定分配成功后）返回指向第一个对象的指针：

```
// 调用 get_size 确定分配多少个 int
int *pia = new int[get_size()]; // pia 指向第一个 int
```

方括号中的大小必须是整型，但不必是常量。

也可以用一个表示数组类型的类型别名（参见 2.5.1 节，第 60 页）来分配一个数组，这样，`new` 表达式中就不需要方括号了：

```
typedef int arrT[42]; // arrT 表示 42 个 int 的数组类型
int *p = new arrT; // 分配一个 42 个 int 的数组；p 指向第一个 int
```

在本例中，`new` 分配一个 `int` 数组，并返回指向第一个 `int` 的指针。即使这段代码中没

有方括号，编译器执行这个表达式时还是会用 `new[]`。即，编译器执行如下形式：

```
int *p = new int[42];
```

分配一个数组会得到一个元素类型的指针

虽然我们通常称 `new T[]` 分配的内存为“动态数组”，但这种叫法某种程度上有些误导。当用 `new` 分配一个数组时，我们并未得到一个数组类型的对象，而是得到一个数组元素类型的指针。即使我们使用类型别名定义了一个数组类型，`new` 也不会分配一个数组类型的对象。在上例中，我们正在分配一个数组的事实甚至都是不可见的——连 `[num]` 都没有。`new` 返回的是一个元素类型的指针。

由于分配的内存并不是一个数组类型，因此不能对动态数组调用 `begin` 或 `end`（参见 3.5.3 节，第 106 页）。这些函数使用数组维度（回忆一下，维度是数组类型的一部分）来返回指向首元素和尾后元素的指针。出于相同的原因，也不能用范围 `for` 语句来处理（所谓的）动态数组中的元素。



要记住我们所说的动态数组并不是数组类型，这是很重要的。

初始化动态分配对象的数组

默认情况下，`new` 分配的对象，不管是单个分配的还是数组中的，都是默认初始化的。可以对数组中的元素进行值初始化（参见 3.3.1 节，第 88 页），方法是在大小之后跟一对空括号。

```
int *pia = new int[10];           // 10 个未初始化的 int
int *pia2 = new int[10]();        // 10 个值初始化为 0 的 int
string *psa = new string[10];    // 10 个空 string
string *psa2 = new string[10](); // 10 个空 string
```

478 在新标准中，我们还可以提供一个元素初始化的花括号列表：

```
C++11 // 10 个 int 分别用列表中对应的初始化器初始化
int *pia3 = new int[10]{0,1,2,3,4,5,6,7,8,9};
// 10 个 string，前 4 个用给定的初始化器初始化，剩余的进行值初始化
string *psa3 = new string[10]{"a", "an", "the", string(3, 'x')};
```

与内置数组对象的列表初始化（参见 3.5.1 节，第 102 页）一样，初始化器会用来初始化动态数组中开始部分的元素。如果初始化器数目小于元素数目，剩余元素将进行值初始化。如果初始化器数目大于元素数目，则 `new` 表达式失败，不会分配任何内存。在本例中，`new` 会抛出一个类型为 `bad_array_new_length` 的异常。类似 `bad_alloc`，此类型定义在头文件 `new` 中。

C++11 虽然我们使用空括号对数组中元素进行值初始化，但不能在括号中给出初始化器，这意味着不能用 `auto` 分配数组（参见 12.1.2 节，第 407 页）。

动态分配一个空数组是合法的

可以用任意表达式来确定要分配的对象数目：

```
size_t n = get_size(); // get_size 返回需要的元素的数目
int* p = new int[n];    // 分配数组保存元素
for (int* q = p; q != p + n; ++q)
    /* 处理数组 */ ;
```

这产生了一个有意思的问题：如果 `get_size` 返回 0，会发生什么？答案是代码仍能正常工作。虽然我们不能创建一个大小为 0 的静态数组对象，但当 `n` 等于 0 时，调用 `new[n]` 是合法的：

```
char arr[0];           // 错误：不能定义长度为 0 的数组
char *cp = new char[0]; // 正确：但 cp 不能解引用
```

当我们用 `new` 分配一个大小为 0 的数组时，`new` 返回一个合法的非空指针。此指针保证与 `new` 返回的其他任何指针都不相同。对于零长度的数组来说，此指针就像尾后指针一样（参见 3.5.3 节，第 106 页），我们可以像使用尾后迭代器一样使用这个指针。可以用此指针进行比较操作，就像上面循环代码中那样。可以从此指针加上（或从此指针减去）0，也可以从此指针减去自身从而得到 0。但此指针不能解引用——毕竟它不指向任何元素。

在我们假想的循环中，若 `get_size` 返回 0，则 `n` 也是 0，`new` 会分配 0 个对象。`for` 循环中的条件会失败（`p` 等于 `q+n`，因为 `n` 为 0）。因此，循环体不会被执行。

释放动态数组

为了释放动态数组，我们使用一种特殊形式的 `delete`——在指针前加上一个空方括号对：

```
delete p;           // p 必须指向一个动态分配的对象或为空
delete [] pa;       // pa 必须指向一个动态分配的数组或为空
```

479

第二条语句销毁 `pa` 指向的数组中的元素，并释放对应的内存。数组中的元素按逆序销毁，即，最后一个元素首先被销毁，然后是倒数第二个，依此类推。

当我们释放一个指向数组的指针时，空方括号对是必需的：它指示编译器此指针指向一个对象数组的第一个元素。如果我们在 `delete` 一个指向数组的指针时忽略了方括号（或者在 `delete` 一个指向单一对象的指针时使用了方括号），其行为是未定义的。

回忆一下，当我们使用一个类型别名来定义一个数组类型时，在 `new` 表达式中不使用 `[]`。即使是这样，在释放一个数组指针时也必须使用方括号：

```
typedef int arrT[42]; // arrT 是 42 个 int 的数组的类型别名
int *p = new arrT;    // 分配一个 42 个 int 的数组；p 指向第一个元素
delete [] p;          // 方括号是必需的，因为我们当初分配的是一个数组
```

不管外表如何，`p` 指向一个对象数组的首元素，而不是一个类型为 `arrT` 的单一对象。因此，在释放 `p` 时我们必须使用 `[]`。



如果我们在 `delete` 一个数组指针时忘记了方括号，或者在 `delete` 一个单一对象的指针时使用了方括号，编译器很可能不会给出警告。我们的程序可能在执行过程中在没有任何警告的情况下行为异常。

智能指针和动态数组

标准库提供了一个可以管理 `new` 分配的数组的 `unique_ptr` 版本。为了用一个 `unique_ptr` 管理动态数组，我们必须在对象类型后面跟一对空方括号：

```
// up 指向一个包含 10 个未初始化 int 的数组
unique_ptr<int[]> up(new int[10]);
up.release(); // 自动用 delete[] 销毁其指针
```

类型说明符中的方括号 (<int[]>) 指出 up 指向一个 int 数组而不是一个 int。由于 up 指向一个数组，当 up 销毁它管理的指针时，会自动使用 delete[]。

指向数组的 unique_ptr 提供的操作与我们在 12.1.5 节（第 417 页）中使用的那些操作有一些不同，我们在表 12.6 中描述了这些操作。当一个 unique_ptr 指向一个数组时，我们不能使用点和箭头成员运算符。毕竟 unique_ptr 指向的是一个数组而不是单个对象，因此这些运算符是无意义的。另一方面，当一个 unique_ptr 指向一个数组时，我们可以使用下标运算符来访问数组中的元素：

```
for (size_t i = 0; i != 10; ++i)
    up[i] = i; // 为每个元素赋予一个新值
```

480

表 12.6: 指向数组的 unique_ptr

指向数组的 unique_ptr 不支持成员访问运算符（点和箭头运算符）。

其他 unique_ptr 操作不变。

unique_ptr<T[]> u	u 可以指向一个动态分配的数组，数组元素类型为 T
unique_ptr<T[]> u(p)	u 指向内置指针 p 所指向的动态分配的数组。p 必须能转换为类型 T*（参见 4.11.2 节，第 143 页）
u[i]	返回 u 拥有的数组中位置 i 处的对象 u 必须指向一个数组

与 unique_ptr 不同，shared_ptr 不直接支持管理动态数组。如果希望使用 shared_ptr 管理一个动态数组，必须提供自己定义的删除器：

```
// 为了使用 shared_ptr，必须提供一个删除器
shared_ptr<int> sp(new int[10], [](int *p) { delete[] p; });
sp.reset(); // 使用我们提供的 lambda 释放数组，它使用 delete[]
```

本例中我们传递给 shared_ptr 一个 lambda（参见 10.3.2 节，第 346 页）作为删除器，它使用 delete[] 释放数组。

如果未提供删除器，这段代码将是未定义的。默认情况下，shared_ptr 使用 delete 销毁它指向的对象。如果此对象是一个动态数组，对其使用 delete 所产生的问题与释放一个动态数组指针时忘记 [] 产生的问题一样（参见 12.2.1 节，第 425 页）。

shared_ptr 不直接支持动态数组管理这一特性会影响我们如何访问数组中的元素：

```
// shared_ptr 未定义下标运算符，并且不支持指针的算术运算
for (size_t i = 0; i != 10; ++i)
    *(sp.get() + i) = i; // 使用 get 获取一个内置指针
```

shared_ptr 未定义下标运算符，而且智能指针类型不支持指针算术运算。因此，为了访问数组中的元素，必须用 get 获取一个内置指针，然后用它来访问数组元素。

12.2.1 节练习

练习 12.23: 编写一个程序，连接两个字符串字面常量，将结果保存在一个动态分配的 char 数组中。重写这个程序，连接两个标准库 string 对象。

练习 12.24: 编写一个程序，从标准输入读取一个字符串，存入一个动态分配的字符数组中。描述你的程序如何处理变长输入。测试你的程序，输入一个超出你分配的数组长度的字符串。

练习 12.25: 给定下面的 new 表达式，你应该如何释放 pa?

```
int *pa = new int[10];
```

12.2.2 allocator 类



new 有一些灵活性上的局限，其中一方面表现在它将内存分配和对象构造组合在了一起。类似的，delete 将对象析构和内存释放组合在了一起。我们分配单个对象时，通常希望将内存分配和对象初始化组合在一起。因为在这种情况下，我们几乎肯定知道对象应有什么值。

481

当分配一大块内存时，我们通常计划在这块内存上按需构造对象。在此情况下，我们希望将内存分配和对象构造分离。这意味着我们可以分配大块内存，但只在真正需要时才真正执行对象创建操作（同时付出一定开销）。

一般情况下，将内存分配和对象构造组合在一起可能会导致不必要的浪费。例如：

```
string *const p = new string[n]; // 构造 n 个空 string
string s;
string *q = p;                  // q 指向第一个 string
while (cin >> s && q != p + n)
    *q++ = s;                   // 赋予*q一个新值
const size_t size = q - p;      // 记住我们读取了多少个 string
// 使用数组
delete[] p; // p 指向一个数组；记得用 delete[] 来释放
```

new 表达式分配并初始化了 n 个 string。但是，我们可能不需要 n 个 string，少量 string 可能就足够了。这样，我们就可能创建了一些永远也用不到的对象。而且，对于那些确实要使用的对象，我们也在初始化之后立即赋予了它们新值。每个使用到的元素都被赋值了两次：第一次是在默认初始化时，随后是在赋值时。

更重要的是，那些没有默认构造函数的类就不能动态分配数组了。

allocator 类

标准库 **allocator** 类定义在头文件 `memory` 中，它帮助我们将内存分配和对象构造分离开来。它提供一种类型感知的内存分配方法，它分配的内存是原始的、未构造的。表 12.7 概述了 **allocator** 支持的操作。在本节中，我们将介绍这些 **allocator** 操作。在 13.5 节（第 464 页），我们将看到如何使用这个类的典型例子。

类似 `vector`，**allocator** 是一个模板（参见 3.3 节，第 86 页）。为了定义一个 **allocator** 对象，我们必须指明这个 **allocator** 可以分配的对象类型。当一个 **allocator** 对象分配内存时，它会根据给定的对象类型来确定恰当的内存大小和对齐位置：

```
allocator<string> alloc;          // 可以分配 string 的 allocator 对象
auto const p = alloc.allocate(n); // 分配 n 个未初始化的 string
```

这个 `allocate` 调用为 n 个 string 分配了内存。

482

表 12.7: 标准库 allocator 类及其算法

<code>allocator<T> a</code>	定义了一个名为 <code>a</code> 的 <code>allocator</code> 对象，它可以为类型为 <code>T</code> 的对象分配内存
<code>a.allocate(n)</code>	分配一段原始的、未构造的内存，保存 <code>n</code> 个类型为 <code>T</code> 的对象
<code>a.deallocate(p, n)</code>	释放从 <code>T*</code> 指针 <code>p</code> 中地址开始的内存，这块内存保存了 <code>n</code> 个类型为 <code>T</code> 的对象； <code>p</code> 必须是一个先前由 <code>allocate</code> 返回的指针，且 <code>n</code> 必须是 <code>p</code> 创建时所要求的大小。在调用 <code>deallocate</code> 之前，用户必须对每个在这块内存中创建的对象调用 <code>destroy</code>
<code>a.construct(p, args)</code>	<code>p</code> 必须是一个类型为 <code>T*</code> 的指针，指向一块原始内存； <code>arg</code> 被传递给类型为 <code>T</code> 的构造函数，用来在 <code>p</code> 指向的内存中构造一个对象
<code>a.destroy(p)</code>	<code>p</code> 为 <code>T*</code> 类型的指针，此算法对 <code>p</code> 指向的对象执行析构函数（参见 12.1.1 节，第 402 页）

allocator 分配未构造的内存

`allocator` 分配的内存是未构造的（`unconstructed`）。我们按需要在此内存中构造对象。在新标准库中，`construct` 成员函数接受一个指针和零个或多个额外参数，在给定位置构造一个元素。额外参数用来初始化构造的对象。类似 `make_shared` 的参数（参见 12.1.1 节，第 401 页），这些额外参数必须是与构造的对象的类型相匹配的合法的初始化器：

C++
11

```
auto q = p; // q 指向最后构造的元素之后的位置
alloc.construct(q++);           // *q 为空字符串
alloc.construct(q++, 10, 'c');  // *q 为 cccccccccc
alloc.construct(q++, "hi");     // *q 为 hi!
```

在早期版本的标准库中，`construct` 只接受两个参数：指向创建对象位置的指针和一个元素类型的值。因此，我们只能将一个元素拷贝到未构造空间中，而不能用元素类型的任何其他构造函数来构造一个元素。

还未构造对象的情况下就使用原始内存是错误的：

```
cout << *p << endl; // 正确：使用 string 的输出运算符
cout << *q << endl; // 灾难：q 指向未构造的内存！
```



为了使用 `allocate` 返回的内存，我们必须用 `construct` 构造对象。使用未构造的内存，其行为是未定义的。

当我们用完对象后，必须对每个构造的元素调用 `destroy` 来销毁它们。函数 `destroy` 接受一个指针，对指向的对象执行析构函数（参见 12.1.1 节，第 402 页）：

483

```
while (q != p)
    alloc.destroy(--q); // 释放我们真正构造的 string
```

在循环开始处，`q` 指向最后构造的元素之后的位置。我们在调用 `destroy` 之前对 `q` 进行了递减操作。因此，第一次调用 `destroy` 时，`q` 指向最后一个构造的元素。最后一步循环中我们 `destroy` 了第一个构造的元素，随后 `q` 将与 `p` 相等，循环结束。



我们只能对真正构造了的元素进行 `destroy` 操作。

一旦元素被销毁后，就可以重新使用这部分内存来保存其他 `string`，也可以将其归

还给系统。释放内存通过调用 `deallocate` 来完成：

```
alloc.deallocate(p, n);
```

我们传递给 `deallocate` 的指针不能为空，它必须指向由 `allocate` 分配的内存。而且，传递给 `deallocate` 的大小参数必须与调用 `allocate` 分配内存时提供的大小参数具有一样的值。

拷贝和填充未初始化内存的算法

标准库还为 `allocator` 类定义了两个伴随算法，可以在未初始化内存中创建对象。表 12.8 描述了这些函数，它们都定义在头文件 `memory` 中。

表 12.8: allocator 算法	
这些函数在给定的位置创建元素，而不是由系统分配内存给它们。	
<code>uninitialized_copy(b,e,b2)</code>	从迭代器 <code>b</code> 和 <code>e</code> 指出的输入范围中拷贝元素到达迭代器 <code>b2</code> 指定的未构造的原始内存中。 <code>b2</code> 指向的内存必须足够大，能容纳输入序列中元素的拷贝
<code>uninitialized_copy_n(b,n,b2)</code>	从迭代器 <code>b</code> 指向的元素开始，拷贝 <code>n</code> 个元素到 <code>b2</code> 开始的内存中
<code>uninitialized_fill(b,e,t)</code>	在迭代器 <code>b</code> 和 <code>e</code> 指定的原始内存范围中创建对象，对象的值均为 <code>t</code> 的拷贝
<code>uninitialized_fill_n(b,n,t)</code>	从迭代器 <code>b</code> 指向的内存地址开始创建 <code>n</code> 个对象。 <code>b</code> 必须指向足够大的未构造的原始内存，能够容纳给定数量的对象

作为一个例子，假定有一个 `int` 的 `vector`，希望将其内容拷贝到动态内存中。我们将分配一块比 `vector` 中元素所占用空间大一倍的动态内存，然后将原 `vector` 中的元素拷贝到前一半空间，对后一半空间用一个给定值进行填充：

```
// 分配比 vi 中元素所占用空间大一倍的动态内存
auto p = alloc.allocate(vi.size() * 2);
// 通过拷贝 vi 中的元素来构造从 p 开始的元素
auto q = uninitialized_copy(vi.begin(), vi.end(), p);
// 将剩余元素初始化为 42
uninitialized_fill_n(q, vi.size(), 42);
```

类似拷贝算法（参见 10.2.2 节，第 341 页），`uninitialized_copy` 接受三个迭代器参数。前两个表示输入序列，第三个表示这些元素将要拷贝到的目的空间。传递给 `uninitialized_copy` 的目的位置迭代器必须指向未构造的内存。与 `copy` 不同，`uninitialized_copy` 在给定的位置构造元素。

类似 `copy`，`uninitialized_copy` 返回（递增后的）目的位置迭代器。因此，一次 `uninitialized_copy` 调用会返回一个指针，指向最后一个构造的元素之后的位置。在本例中，我们将此指针保存在 `q` 中，然后将 `q` 传递给 `uninitialized_fill_n`。此函数类似 `fill_n`（参见 10.2.2 节，第 340 页），接受一个指向目的位置的指针、一个计数和一个值。它会在目的位置指针指向的内存中创建给定数目个对象，用给定值对它们进行初始化。

12.2.2 节练习

练习 12.26: 用 `allocator` 重写第 427 页中的程序。



12.3 使用标准库：文本查询程序

我们将实现一个简单的文本查询程序，作为标准库相关内容学习的总结。我们的程序允许用户在一个给定文件中查询单词。查询结果是单词在文件中出现的次数及其所在行的列表。如果一个单词在一行中出现多次，此行只列出一行。行会按照升序输出——即，第 7 行会在第 9 行之前显示，依此类推。

例如，我们可能读入一个包含本章内容（指英文版中的文本）的文件，在其中寻找单词 `element`。输出结果的前几行应该是这样的：

```
element occurs 112 times
(line 36) A set element contains only a key;
(line 158) operator creates a new element
(line 160) Regardless of whether the element
(line 168) When we fetch an element from a map, we
(line 214) If the element is not found, find returns
```

接下来还有大约 100 行，都是单词 `element` 出现的位置。



12.3.1 文本查询程序设计

485

开始一个程序的设计的一种好方法是列出程序的操作。了解需要哪些操作会帮助我们分析出需要什么样的数据结构。从需求入手，我们的文本查询程序需要完成如下任务：

- 当程序读取输入文件时，它必须记住单词出现的每一行。因此，程序需要逐行读取输入文件，并将每一行分解为独立的单词
- 当程序生成输出时，
 - 它必须能提取每个单词所关联的行号
 - 行号必须按升序出现且无重复
 - 它必须能打印给定行号中的文本。

利用多种标准库设施，我们可以很漂亮地实现这些要求：

- 我们将使用一个 `vector<string>` 来保存整个输入文件的一份拷贝。输入文件中的每行保存为 `vector` 中的一个元素。当需要打印一行时，可以用行号作为下标来提取行文本。
- 我们使用一个 `istringstream`（参见 8.3 节，第 287 页）来将每行分解为单词。
- 我们使用一个 `set` 来保存每个单词在输入文本中出现的行号。这保证了每行只出现一次且行号按升序保存。
- 我们使用一个 `map` 来将每个单词与它出现的行号 `set` 关联起来。这样我们就可以方便地提取任意单词的 `set`。

我们的解决方案还使用了 `shared_ptr`，原因稍后进行解释。

数据结构

虽然我们可以用 `vector`、`set` 和 `map` 来直接编写文本查询程序，但如果定义一个更